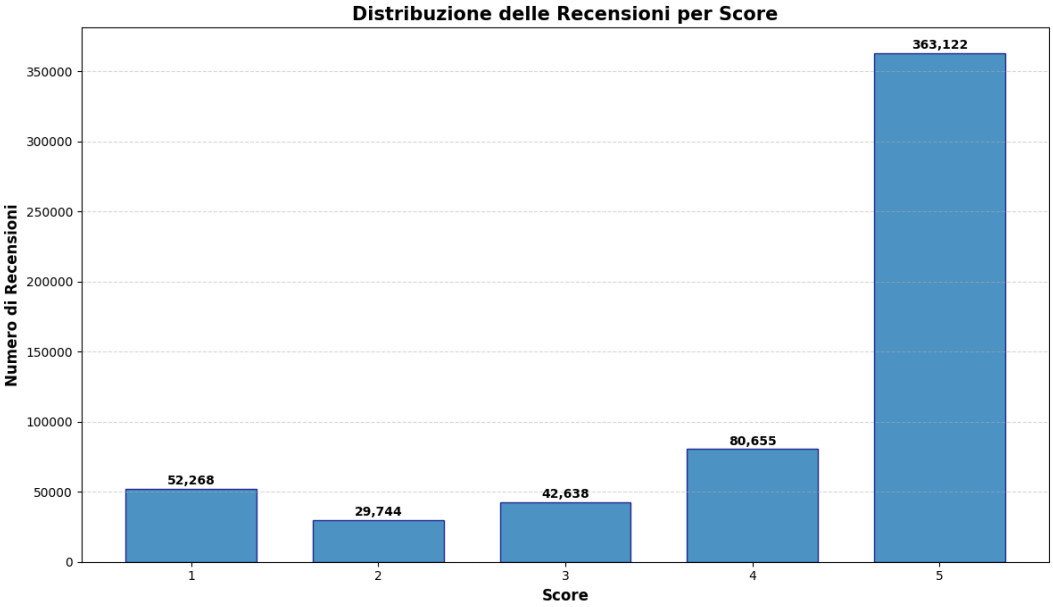
Sistemi Informativi Evoluti e Big Data – A.A. 2025-2026

Homework #1 – MongoDB

1. Interrogazioni Analitiche

1. Trovare il numero di recensioni per ciascun valore di score e visualizzarlo in un istogramma



2. Trovare i 5 prodotti con la media di score più alta

1. Product ID: B000RL53PE
2. Product ID: B005EL6VOY
3. Product ID: B001IZ9ME6
4. Product ID: B00376ZEY6
5. Product ID: B0029O0XGQ

3. Trovare i 5 utenti che hanno scritto più recensioni

1. C. F. Hill “CFH" -> 448

2. O. Brown "Ms. O. Khannah-Brown” -> 421

3. Gary Peterson -> 389

4.Rebecca of Amazon "The Rebecca Review” -> 365

5. c2 -> 256

4. Calcolare il numero di recensioni positive, neutre e negative per ogni valore di score (usando l’attributo sentiment)

Score	Positive	Neutral	Negative	Total
1	16,448	14,468	21,352	52,268
2	13,629	8,381	7,734	29,744
3	26,189	10,480	5,969	42,638
4	66,051	10,955	3,649	80,655
5	320,939	33,434	8,749	363,122

5. Trovare i prodotti che hanno sia un alto punteggio medio (score >= 4.5) che recensioni con almeno il 10% di sentiment negativo

Product ID	Avg Score	Reviews	Neg	Neg %
B000FBMFDO	4.62	97	57	58.8%
B000EPOBYC	4.54	13	4	30.8%
B000WAFJ0E	4.50	10	3	30.0%
B0029J6JI2	4.70	10	3	30.0%
B001JU7YJA	4.50	10	3	30.0%

2. Discussione sull'Architettura del Database

La scelta di suddividere le recensioni in sei collezioni distinte (score_1...score_5 e no_text) si è rivelata un ostacolo complesso per lo svolgimento delle query. Se da un lato questa struttura ha senso per isolare i dati per un singolo score, ha complicato qualsiasi analisi che richiedesse una visione d'insieme.

a) Impatto sulla Complessità delle Query

La suddivisione ha aumentato la complessità di quasi tutte le query analitiche.

Query Semplici (Query 1 e 4): Per rispondere a domande "per score" è stata implementata la logica in Python. Sono state eseguite 5 query di aggregazione separate (una per collezione) e poi uniti i risultati manualmente nello script per generare la tabella e il grafico.

Query Complesse (Query 2, 3 e 5): Per le query che dovevano analizzare tutti i dati (top prodotti, top utenti), la complessità è esplosa. L'unica soluzione performante lato database è stata l'uso dell'operatore *$unionWith*. Questo ha reso le pipeline macchinose, richiedendo 4-5 stage di unione prima di poter iniziare la vera logica analitica (*$group, $match*).

Con una collezione unica *reviews*, ogni query sarebbe stata una singola pipeline di aggregazione, più semplice da scrivere, leggere e manutenere.

b) Impatto su Prestazioni e Risorse

Le prestazioni sono calate a causa di questa scelta, sia lato client che lato database.

Overhead delle Query: L'esecuzione di 5 query separate (Query 1 e 4) implica 5 round-trip di rete tra l'applicazione e il database, un overhead non necessario.

Overhead di $unionWith: L'uso di *$unionWith* (Query 2, 3, 5) non è a costo zero. Obbliga MongoDB a gestire e unire 5-6 flussi di dati diversi prima di procedere. Questo è più lento e richiede più CPU e RAM lato server rispetto a una singola aggregazione su una collezione unica.

Vantaggio Annullato: L'unico vantaggio sarebbe un filtering automatico per score (collezione = score). Tuttavia, le query richieste hanno dimostrato che quasi tutte le analisi cercavano dati da tutte le collezioni.

c) Impatto su Manutenibilità e Scalabilità

Questo è l'aspetto più problematico a lungo termine.

Manutenibilità:

Schema: Qualsiasi modifica allo schema (es. aggiungere un campo) deve essere replicata su TUTTE le collezioni.

Indici: Ogni indice necessario (su *product_id*, *user_id*, *time*) deve essere creato e mantenuto su TUTTE le collezioni, aumentando l'uso dello spazio su disco e il costo di ogni operazione di scrittura.

Scalabilità:

Sharding: Tentare di partizionare orizzontalmente basandosi sullo score sarebbe un disastro. Si creerebbe un "hot shard", poiché la collezione *score_5* (con ~65% dei dati) riceverebbe la maggior parte del traffico.

Crescita: Con la crescita del dataset, la gestione di tutte le collezioni diventerebbe molto complessa.

3. Confronto con una Collezione Unica Indicizzata

Una singola collezione *reviews* è la soluzione preferibile. La strategia corretta non è il partizionamento fisico (collezioni multiple), ma il partizionamento logico tramite indici.

Mantenendo tutti i ~568.000 documenti in un'unica collezione reviews, si potrebbero creare indici sui campi *score*, *product_id* e *sentimentv*per ottimizzare le query.

Vantaggi:

Le query sarebbero più semplici (pipeline singole, senza $unionWith)

Le performance aumenterebbero grazie all'uso ottimale degli indici

La manutenibilità sarebbe banale (uno schema, un set di indici)

Quando usare Collezioni Multiple (Casi Utili):

- Dati "Caldi" vs "Freddi": Separare dati recenti da quelli storici

- Diverse Politiche di Retention (TTL): Collezioni con scadenze diverse

- Isolamento del Working Set: Dati di "tenant" diversi mai aggregati

- Schema Davvero Diverso: Documenti che rappresentano entità diverse

Quando NON usare Collezioni Multiple:

- Dati con schema identico

- Query analitiche/aggregative tra i "silos"

- Partizionamento su campi a bassa cardinalità (come score con 5 valori)

4. Classificazione del Sistema di Raccomandazione

Il sistema implementato è un Sistema di Raccomandazione Ibrido (Hybrid Recommender System).

Utilizza una strategia ibrida pesata (weighted hybrid), che combina i punteggi di tre diverse tecniche per produrre un'unica classifica finale:

1. Collaborative Filtering (Filtro Collaborativo):
Metodo: co-occorrenza di utenti.

Perché: Misura la rilevanza specifica. Raccomanda prodotti basandosi sul comportamento di utenti simili (la "saggezza della folla"). Risponde alla domanda: "Quali altri prodotti sono rilevanti per le persone a cui è piaciuto il prodotto target?".

2. Content-Based Filtering (Filtro Basato sul Contenuto):

Questo approccio misura la qualità intrinseca del prodotto, utilizzando due diversi attributi (contenuti):

Metodo (Testo): somiglianza lessicale (TF-IDF e cosine similarity).

Perché: Misura la similarità tematica. Risponde alla domanda: "Quali altri prodotti sono tematicamente simili al prodotto target?".

Metodo (Qualità): media dei sentiment.

Perché: Misura la qualità globale. Agisce come un filtro, rispondendo alla domanda: "Tra i prodotti candidati, quali sono effettivamente apprezzati dal pubblico generale?”.

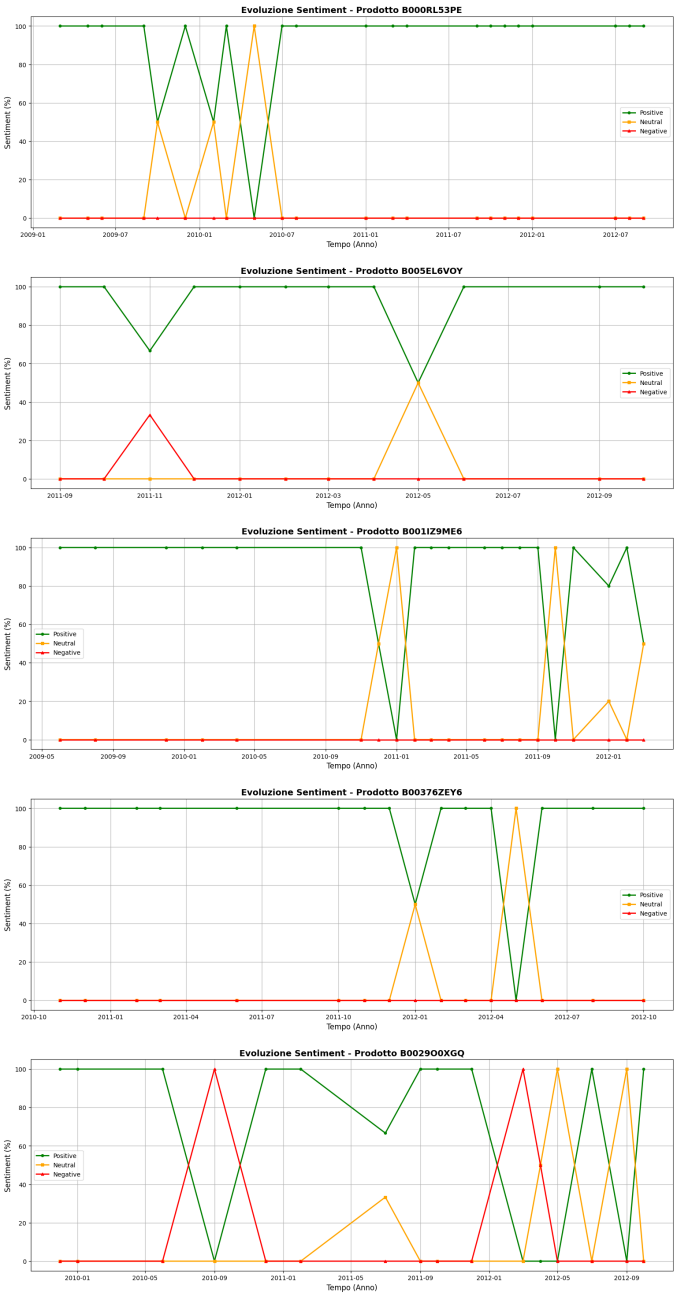
5. Analisi Avanzata – Cause delle Variazioni e Adattamento del Sistema

Analisi delle Cause: Rumore Statistico, non Trend

L'analisi dei 5 prodotti "top" rivela un pattern chiaro: non si tratta di prodotti popolari, ma di prodotti sporadici che hanno ricevuto pochissime recensioni (tutte con punteggio 5.0).

Osservando i grafici e l'elenco delle anomalie, si notano cambiamenti estremi: POSITIVE DOWN 100.0% seguito da POSITIVE UP 100.0%.

Causa Primaria (Basso Volume): Questa volatilità non è un cambiamento nel trend di mercato, ma un artefatto statistico causato da un denominatore piccolissimo. Se un prodotto riceve una sola recensione positiva in un mese, il sentiment è 100% positivo; se nel mese successivo riceve una recensione negativa, il sentiment crolla a 0% positivo.



Causa Secondaria (Prodotti di Nicchia): Si tratta probabilmente di prodotti specifici acquistati e recensiti molto di rado.

Come il Sistema di Raccomandazione Dovrebbe Adattarsi

Adattamento 1: Introdurre la "Fiducia" (Confidence)

Il sistema ibrido potrebbe erroneamente promuovere prodotti solo perché hanno un avg_score di 5.0 con poche recensioni.

Soluzione: Sostituire la media semplice con un punteggio bayesiano o Wilson Score, che "sconta" il punteggio dei prodotti con poche recensioni tirandoli verso la media globale del dataset.

Impatto: Un prodotto 5.0 con 10 recensioni verrebbe classificato più in basso di un 4.8 con 5000 recensioni.

Adattamento 2: Soglia Minima di Volume per le Anomalie

Lo script attuale è "rumoroso" perché non distingue tra cambiamenti basati su 2 o 2000 recensioni.

Soluzione: Modificare *detect_sentiment_anomalies* per attivarsi solo se (variazione > soglia) AND (volume_mensile > N) (es. N > 20).

Impatto: Filtra il 90% delle anomalie "rumorose", facendo emergere solo i veri cambiamenti di trend.

Adattamento 3: Penalizzazione Temporale

Applicare una penalizzazione sul *final_score* solo se un prodotto mostra un trend negativo statisticamente significativo.

Impatto: Il sistema ignorerebbe il rumore dei prodotti di nicchia, ma reagirebbe rapidamente a un prodotto popolare che riceve un'ondata di recensioni negative reali.